```
Feature: Frequent Flyer status is calculated based on points
  As a Frequent Flyer member
  I want my status to be upgraded as soon as I earn enough points
  So that I can benefit from my higher status sooner
```

Recall the business goals behind this requirement.

```
Scenario: New members should start out as BRONZE members
  Given Jill Smith is not a Frequent Flyer member
  When she registers on the Frequent Flyer program
  Then she should have a status of BRONZE
```

This scenario describes what status members should get when they start out.

```
Scenario Outline:
  Given Joe Jones is a <initialStatus> Frequent Flyer member
  And he has <initialStatusPoints> status points
  When he earns <extraPoints> extra status points
  Then he should have a status of <finalStatus>
```

This scenario describes the actual status earning process.

```
  Examples: Status points required for each level
    | initialStatus | initialStatusPoints | extraPoints | finalStatus |
    | Bronze        | 0                   | 300         | Silver      |
    | Bronze        | 100                 | 200         | Silver      |
    | Silver        | 0                   | 700         | Gold        |
    | Gold          | 0                   | 1500        | Platinum    |
```

Illustrate the number of points needed for each level.

This is a simple tangible business requirement with a clear value proposition and two acceptance criteria. Still, it's probably too big to write in one pass, so you'd typically implement the individual acceptance criteria one at a time. In any case, the first step is to automate the step definitions for these scenarios.

10.2.3 Automate the acceptance criteria scenarios

The first acceptance criterion describes the initial status level of a new Frequent Flyer member:

```
Scenario: New members should start out as BRONZE members
  Given Jill Smith is not a Frequent Flyer member
  When she registers on the Frequent Flyer program
  Then she should have a status of BRONZE
```

Background and context — **The behavior under test** — **The expected outcome**

You could automate this scenario using the techniques we discussed in the previous chapters. In Cucumber, for example, the step definitions might look something like this:

Background and context

```
@Given("^(\\S*) (\\S*) is not a Frequent Flyer member$")
public void not_a_Frequent_Flyer_member(String name) throws Throwable {...}
```

The behavior under test

```
@When("^(?:s?)he registers on the Frequent Flyer program$")
public void registers_on_the_Frequent_Flyer_program() throws Throwable {...}
```

The expected outcome

```
@Then("^(?:s?)he should have a status of (.*)$")
public void should_have_status_of(FrequentFlyerStatus expectedStatus) {...}
```

But it's when you actually implement the step definitions that you really discover what code you need to write.